



Politechnika Łódzka

Instytut Automatyki

Zakład sterowania robotów

TWORZENIE SYSTEMÓW ROBOTYCZNYCH

Temat 6

Wizualizacja i rejestrowanie danych

14 marca 2025



Spis treści

1	Wprowadzenie	2
2	Narzędzia RQT w ROS2	2
2.1	RQT_GUI	2
2.2	RQT_MSG	3
2.3	RQT_TOPIC	4
2.4	RQT_PUBLISHER	4
2.5	RQT_SRV	5
2.6	RQT_SERVICE_CALLER	5
2.7	RQT_ACTION	6
2.8	RQT_RECONFIGURE	6
2.9	RQT_GRAPH	7
2.10	RQT_CONSOLE	7
2.11	RQT_PLOT	8
2.12	RQT_IMAGE_VIEW	9
2.13	RQT_TF_TREE	9
2.14	RQT_BAG	10
3	Wizualizacja danych - RViz	10
4	Wizualizacja i Analiza Danych - PlotJuggler	11
5	Zegary	12
5.1	Rodzaje zegarów	12
5.2	Źródło czasu	13
5.3	Implementacja	13
6	Nagrywanie i odtwarzanie danych - Bag	13
6.1	Rejestrowanie danych - <code>ros2 bag record</code>	14
6.1.1	Podstawowe polecenia	14
6.1.2	Zastosowanie filtracji tematów przy rejestrowaniu tematów	14
6.2	Podstawowe informacje o bagu - <code>ros2 bag info</code>	15
6.3	Odtwarzanie baga - <code>ros2 bag play</code>	15
7	Logowanie danych diagnostycznych - Logging	16
8	Dokumentacja i inne linki	17

1 Wprowadzenie

W ROS2 dostępnych jest wiele narzędzi wspomagających rozwój, debugowanie i analizę systemów robotycznych. Podstawowe dostępne zestawy narzędzi to RQT, RViz, PlotJuggler oraz ROS2 Bag.

RQT to zbiór aplikacji graficznych opartych na Qt, które umożliwiają wizualizację, monitorowanie i konfigurację ROS2 w sposób interaktywny. Dzięki narzędziom RQT można w łatwy sposób analizować strukturę komunikacji między węzłami, podglądać wiadomości przesyłane w tematach, a także modyfikować parametry działających komponentów systemu.

RViz (ROS Visualization) to potężne narzędzie graficzne, które umożliwia użytkownikom ROS2 wyświetlanie danych z różnych sensorów, analizowanie transformacji układów współrzędnych oraz wizualizację trajektorii i map.

PlotJuggler to narzędzie graficzne przeznaczone do analizy i wizualizacji danych numerycznych w ROS2. Umożliwia ono dynamiczne wyświetlanie i przetwarzanie danych strumieniowych.

Czas w ROS2 (Clock) jest kluczowy dla synchronizacji danych z różnych czujników, takich jak LIDAR, kamery czy IMU, co jest niezbędne do poprawnego działania algorytmów wykorzystujących fuzję danych z czujników i sterowania. W symulacjach umożliwia precyzyjne odtwarzanie scenariuszy testowych i analizowanie zachowania systemu. Dodatkowo, poprawna obsługa czasu jest istotna dla analizy danych historycznych z `rosbag2`, zapewniając spójność w rejestrowaniu i odtwarzaniu zdarzeń, co umożliwia dokładniejszą diagnostykę przyczyn błędów. Przyspieszanie, zatrzymywanie lub cofanie czasu jest wykorzystywane podczas odtwarzania zebranych danych.

ROS2 Bag to natomiast zestaw narzędzi przeznaczonych do nagrywania i odtwarzania danych w ROS2. Umożliwia ono rejestrowanie przesyłanych wiadomości, co jest niezwykle przydatne do analizy, testowania algorytmów oraz diagnostyki systemów robotycznych. Dzięki mechanizmowi odtwarzania można symulować działanie robota lub innych urządzeń na podstawie wcześniej zebranych danych, co ułatwia rozwój i testowanie algorytmów bez konieczności posiadania lub dostępu do rzeczywistego robota.

Moduł Logging w ROS2 służy do zarządzania logowaniem wiadomości w systemie ROS, umożliwiając monitorowanie działania węzłów, diagnozowanie błędów oraz analizę działania systemu.

2 Narzędzia RQT w ROS2

RQT to zbiór narzędzi GUI w ROS2 opartych na Qt, które pomagają w monitorowaniu, debugowaniu i analizie systemu. Każde z narzędzi ma określone zastosowanie, które ułatwia pracę z ROS2, umożliwiając np. podgląd tematów, wiadomości, transformacji TF czy zmianę parametrów węzłów.

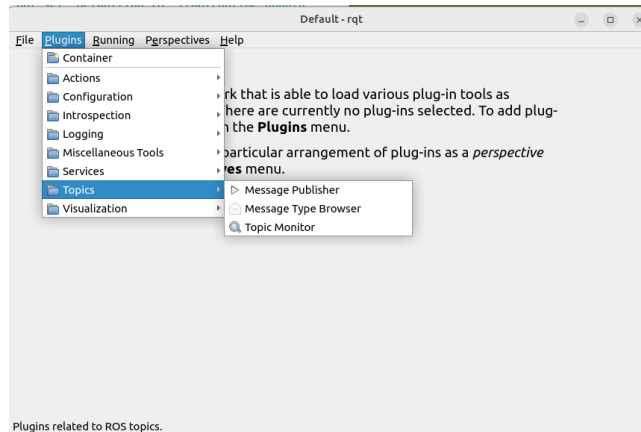
2.1 RQT_GUI

RQT GUI jest głównym interfejsem graficznym umożliwiającym łączenie różnych narzędzi RQT w jednym oknie. Dzięki temu użytkownik może stworzyć własne środowisko monitorowania, np. połączyć widok tematów, wiadomości i wykresów w jednym interfejsie.

Umożliwia utworzenie wielu różnych widoków i przełączanie się między nimi (**Perspectives**). Z listy rozwi-

janej Plugins można wybrać i umieścić w widoku dowolne narzędzie RQT.

```
1 ros2 run rqt_gui rqt_gui
```



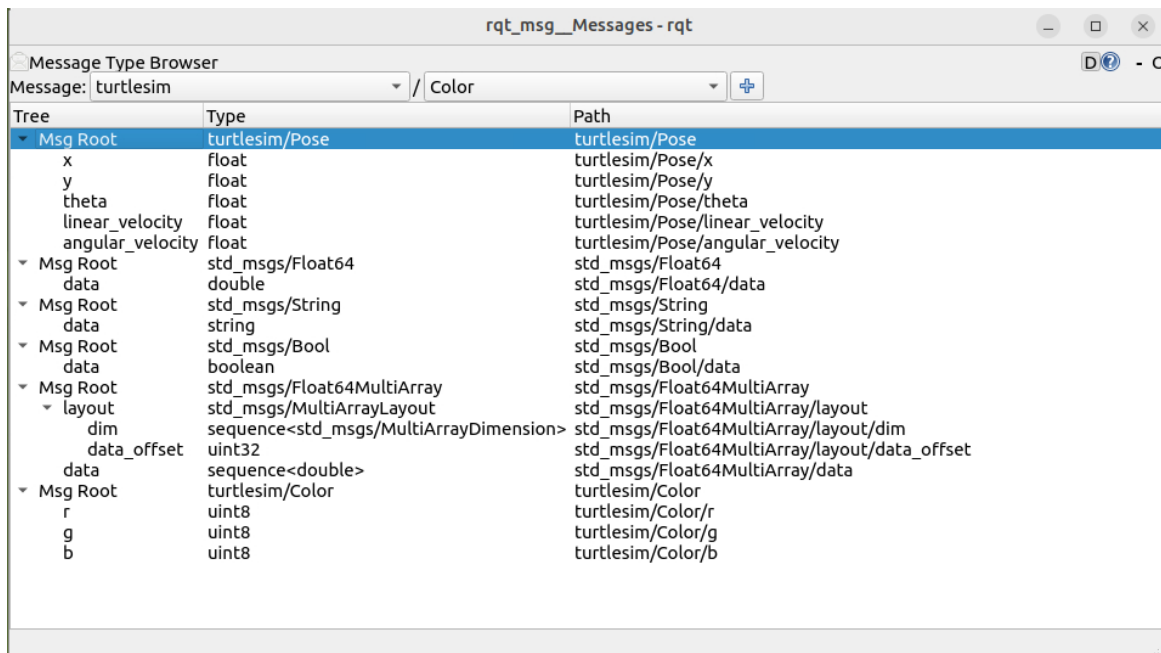
Rys. 2.1: GUI dla narzędzia RQT_GUI

2.2 RQT_MSG

Graficzne narzędzie do podglądu zainstalowanych w systemie typów wiadomości używanych w ROS2 Topic. Jest przydatne do analizy struktury danych przesyłanych między węzłami. W pierwszej liście rozwijanej od lewej znajduje się lista dostępnych pakietów zawierających wiadomości dla ROS Topic. Natomiast druga lista rozwijana zawiera odpowiednie typy wiadomości z pakietów. Po dodaniu wiadomości (+) pojawia się ona na liście wraz z nazwami pól i ich typami.

Uruchomienie narzędzia:

```
1 ros2 run rqt_msg rqt_msg
```



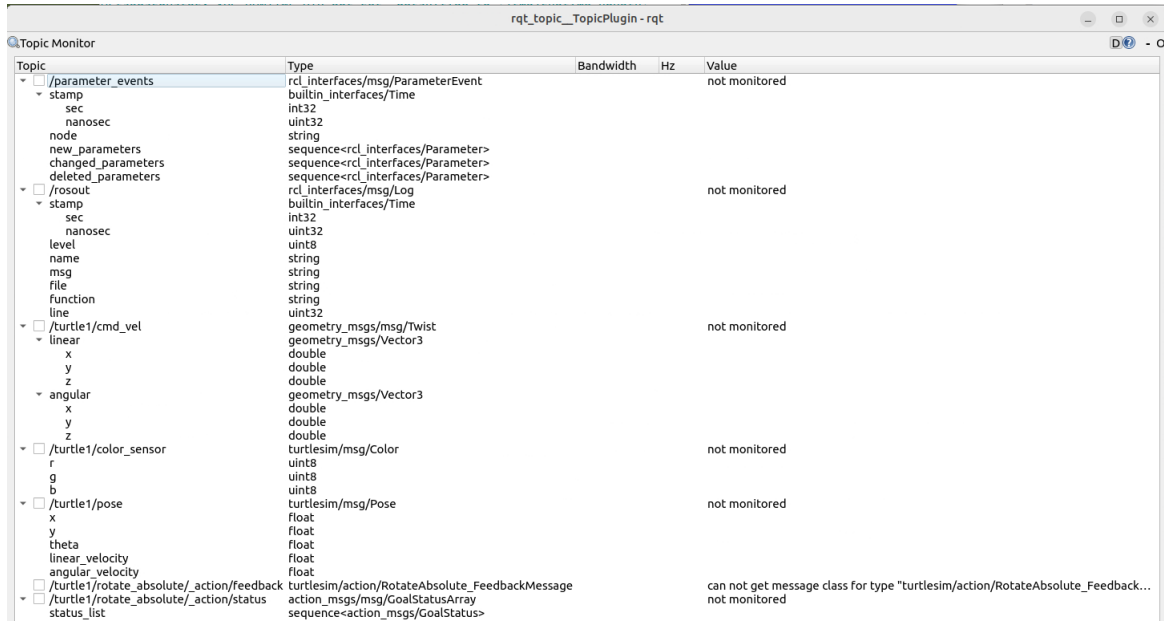
Rys. 2.2: GUI dla narzędzia RQT_MSG

2.3 RQT_TOPIC

RQT_TOPIC umożliwia monitorowanie aktywnych tematów w ROS2. Można zobaczyć listę dostępnych tematów oraz analizować ich częstotliwość publikacji i przesyłane dane. Narzędzie wyświetla wszystkie tematy jakie są w systemie i działa jak Subscriber. Po zaznaczeniu tematów, które chcemy monitorować wyświetlane są odbierane na nich wartości wysyłane przez istniejące węzły z Publisherami.

Uruchomienie narzędzia:

```
1 ros2 run rqt_topic rqt_topic
```



Rys. 2.3: GUI dla narzędzia RQT_TOPIC

2.4 RQT_PUBLISHER

Narzędzie do publikowania wiadomości na określone tematy w ROS2. Użytkownik może określić typ wiadomości, częstotliwość wysyłania oraz wysłać pojedynczą wiadomość testową. Przydatne do testowania interakcji między węzłami. Narzędzie umożliwia wybór tematu z listy dostępnych tematów lub dodanie nowego (wpisanie własnej nazwy tematu w miejsce Topic), jeśli chcemy utworzyć nowego Publishera.

Uruchomienie narzędzia:

```
1 ros2 run rqt_publisher rqt_publisher
```

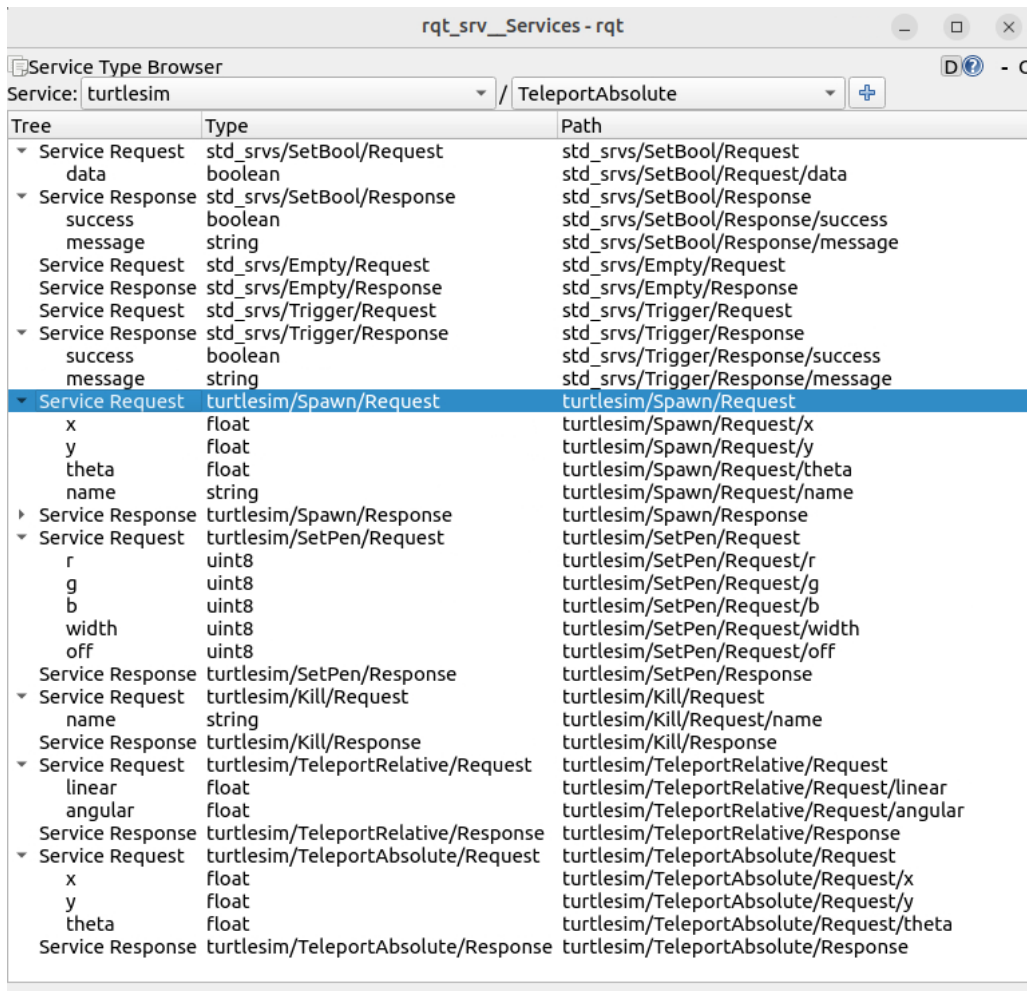


Rys. 2.4: GUI dla narzędzia RQT_PUBLISHER

2.5 RQT_SRV

Narzędzie pozwala na podgląd typów wiadomości serwisowych, zarówno dla zapytań (request), jak i odpowiedzi (response). Jest pomocne przy pracy z serwisami ROS2. W pierwszej liście rozwijanej od lewej znajduje się lista dostępnych pakietów zawierających wiadomości serwisowe dla ROS Service. Natomiast druga lista rozwijana zawiera odpowiednie typy wiadomości serwisowych z pakietów. Po dodaniu wiadomości (+) pojawia się ona na liście wraz z nazwami pól i ich typami. Dodatkowo występuje rozdzielenie na żądanie (*ServiceRequest*) i odpowiedź (*ServiceResponse*).

```
1 ros2 run rqt_srv rqt_srv
```

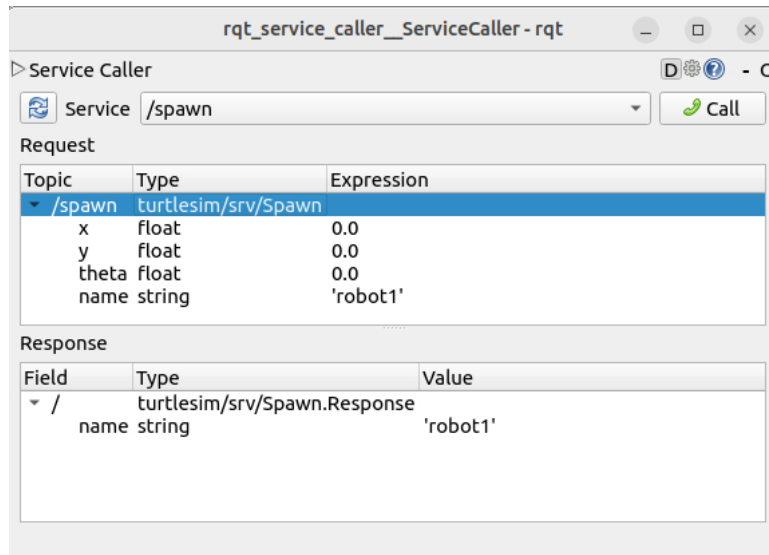


Rys. 2.5: GUI dla narzędzia RQT_SRV

2.6 RQT_SERVICE_CALLER

Narzędzie umożliwia interaktywne wywoływanie serwisów w ROS2, tzn. działa jak klient. Jest szczególnie przydatne do testowania komunikacji klient-serwer. Z listy dostępnych serwisów wybierany jest jeden. W sekcji żądania (*Request*) uzupełnia się pola z wartościami a po wywołaniu `call` otrzymuje się odpowiedź.

```
1 ros2 run rqt_service_caller rqt_service_caller
```



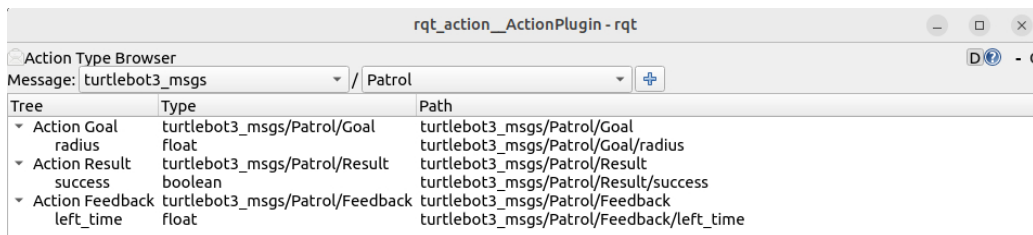
Rys. 2.6: GUI dla narzędzia RQT_SERVICE_CALLER

2.7 RQT_ACTION

RQT_ACTION umożliwia graficzny podgląd struktury wiadomości używanych w akcjach ROS2. Podobnie jak RQT_MSG i RQT_SRV, pozwala analizować strukturę danych, ale w tym przypadku skupia się na akcjach, które składają się z trzech warstw: goal (cel), feedback (informacja zwrotna) i result (wynik).

W pierwszej liście rozwijanej od lewej znajduje się lista dostępnych pakietów zawierających wiadomości dla ROS ACTION. Natomiast druga lista rozwijana zawiera odpowiednie typy wiadomości dla akcji z pakietów. Po dodaniu wiadomości (+) pojawia się ona na liście wraz z nazwami pól i ich typami.

```
1 ros2 run rqt_action rqt_action
```

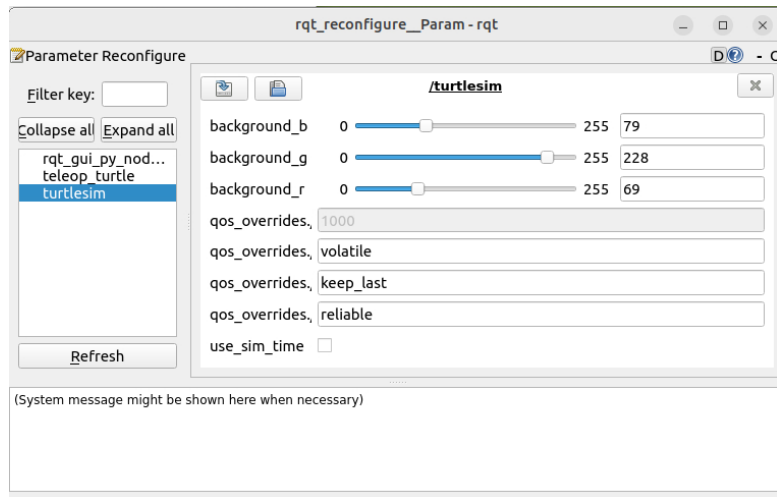


Rys. 2.7: GUI dla narzędzia RQT_ACTION

2.8 RQT_RECONFIGURE

RQT_RECONFIGURE pozwala na dynamiczną zmianę parametrów węzłów ROS2 w czasie rzeczywistym. Jest szczególnie przydatne w systemach, gdzie konieczne jest dostosowanie parametrów bez konieczności restartowania węzła lub wpisywania polecenia zmiany wartości parametru w terminalu, np. w regulacji PID, dostrajaniu algorytmów sterowania czy konfiguracji czujników. Z lewej strony znajduje się lista dostępnych węzłów. Po wybraniu interesujących węzłów z prawej strony pojawiają się dla nich parametry wraz z dopuszczalnymi zakresami zmian.

```
1 ros2 run rqt_reconfigure rqt_reconfigure
```

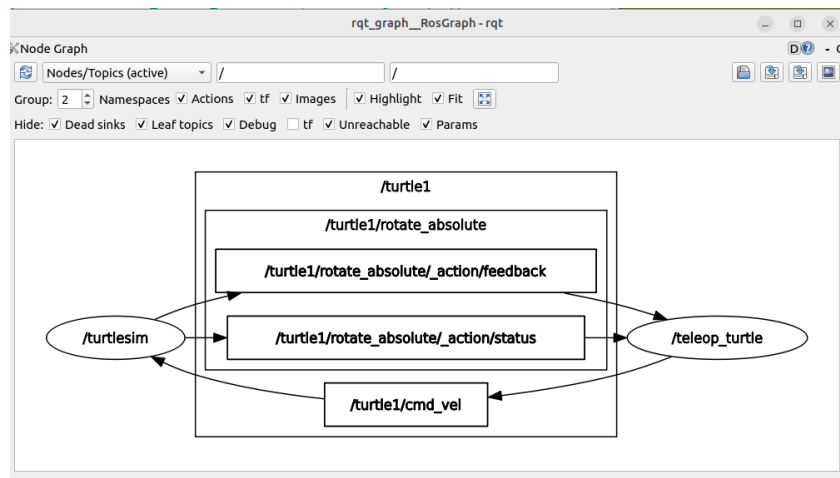


Rys. 2.8: GUI dla narzędzia RQT_RECONFIGURE

2.9 RQT_GRAPH

RQT_GRAPH umożliwia wizualizację grafu ROS2, pokazując powiązania między węzłami, tematami i serwisami. Jest przydatne do debugowania zależności między komponentami.

```
1 ros2 run rqt_graph rqt_graph
```



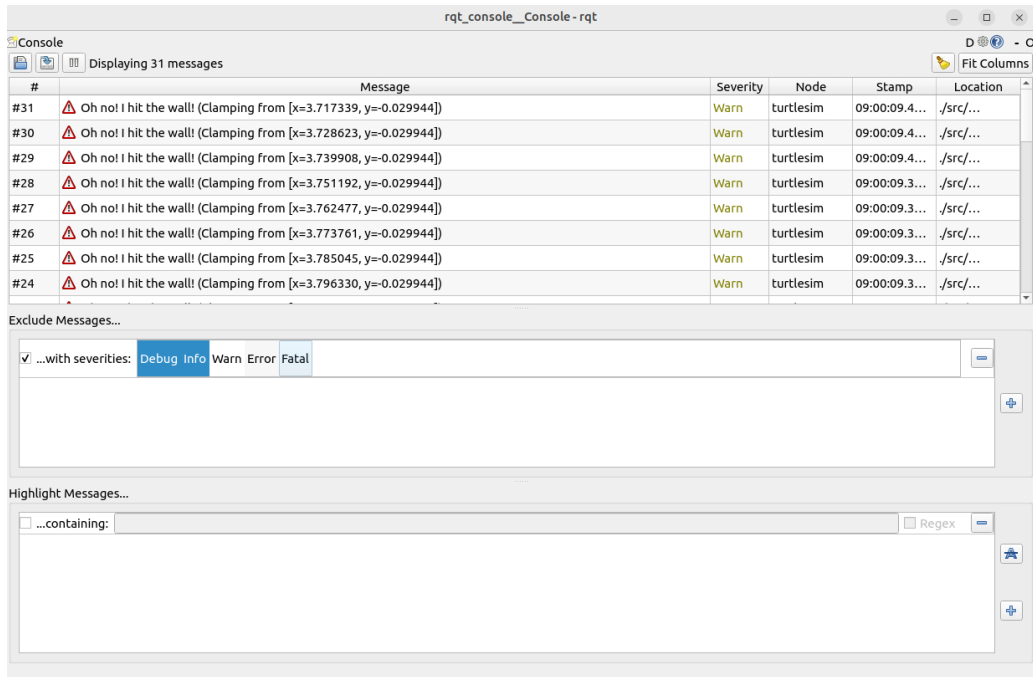
Rys. 2.9: GUI dla narzędzia RQT_GRAPH

2.10 RQT_CONSOLE

RQT_CONSOLE to narzędzie umożliwiające monitorowanie logów systemowych generowanych przez węzły ROS2. Pozwala ono filtrować wiadomości według poziomów logowania (INFO, WARN, ERROR, DEBUG), co jest niezwykle pomocne podczas debugowania działania systemu. Dzięki temu można łatwo analizować, które węzły generują błędy, ostrzeżenia lub istotne informacje diagnostyczne.

Jest to narzędzie szczególnie użyteczne w dużych systemach ROS2, gdzie monitorowanie logów w terminalu staje się nieefektywne. Możliwość przeszukiwania i sortowania logów znacznie ułatwia diagnostykę błędów w aplikacjach robotycznych.

```
1 ros2 run rqt_console rqt_console
```

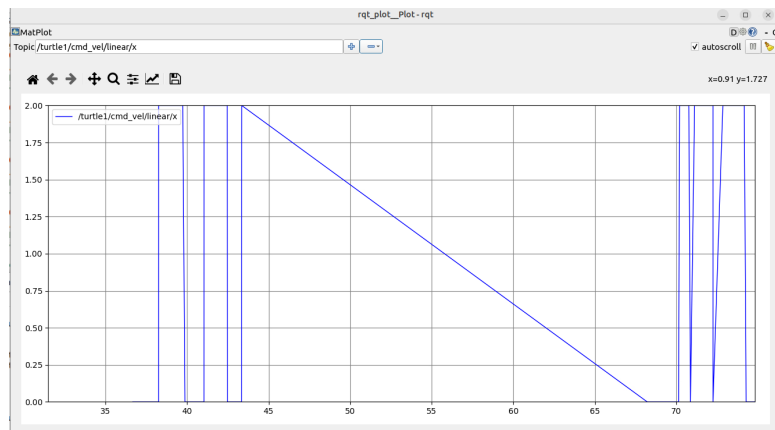


Rys. 2.10: GUI dla narzędzia RQT_CONSOLE

2.11 RQT_PLOT

Umożliwia wizualizację danych numerycznych publikowanych na tematach ROS2 w formie wykresów. Jest szczególnie użyteczne w analizie danych z czujników. Do wyboru wyświetlanych wartości należy wpisać nazwę tematu wraz z kolejnymi polami w wiadomości, które mają być wyświetlane. Istnieje możliwość dodania wielu wartości do wykresu. Przykładowo dla wiadomości typu `Twist` od prędkości po dodaniu pola `linear` do wykresu zostaną dodane 3 wartości od prędkości w kierunku x, y, z.

```
1 ros2 run rqt_plot rqt_plot
```

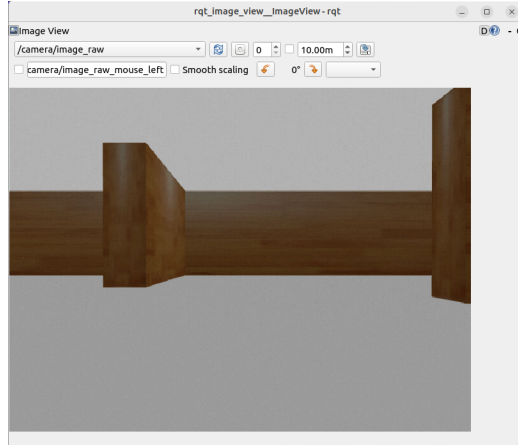


Rys. 2.11: GUI dla narzędzia RQT_PLOT

2.12 RQT_IMAGE_VIEW

Narzędzie do podglądu obrazów przesyłanych w ROS2, np. z kamer lub strumieni wideo.

```
1 ros2 run rqt_image_view rqt_image_view
```

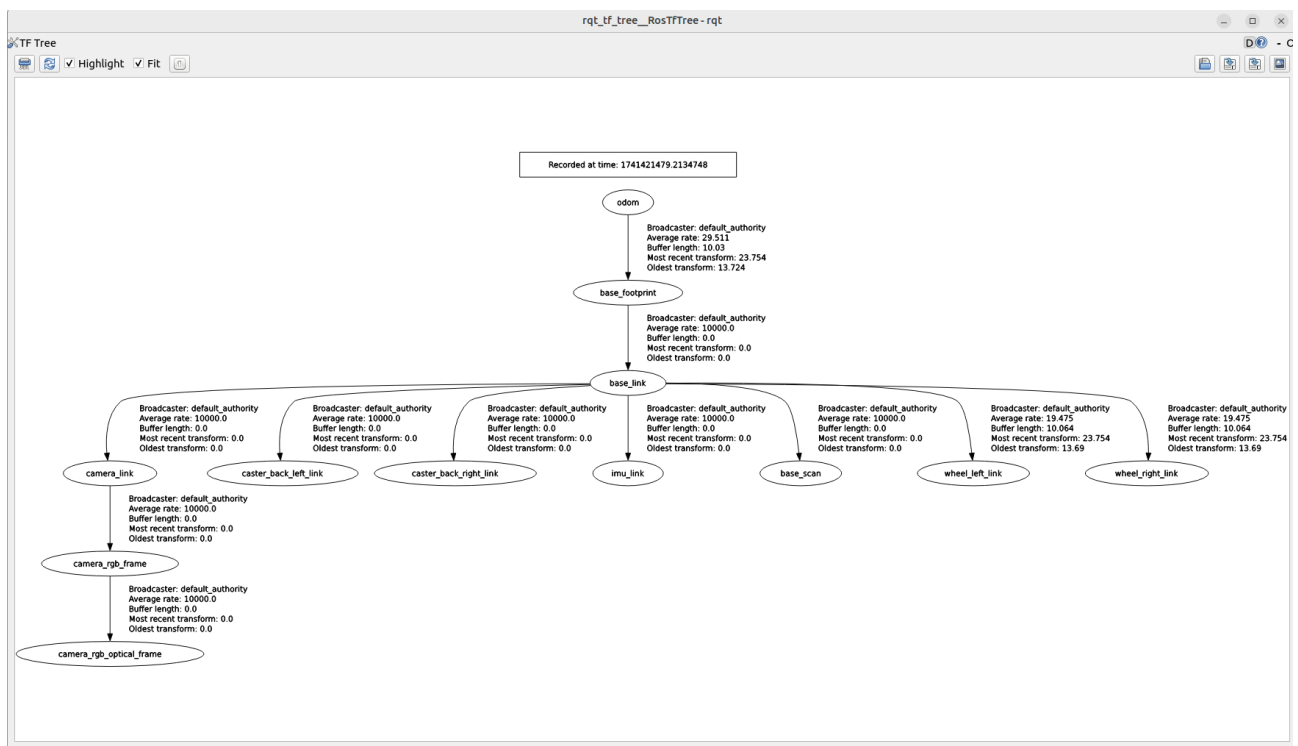


Rys. 2.12: GUI dla narzędzia RQT_IMAGE_VIEW

2.13 RQT_TF_TREE

Umożliwia wizualizację transformacji układów współrzędnych w systemie ROS2. Jest używane w analizie nawigacji i orientacji robota w przestrzeni.

```
1 ros2 run rqt_tf_tree rqt_tf_tree
```



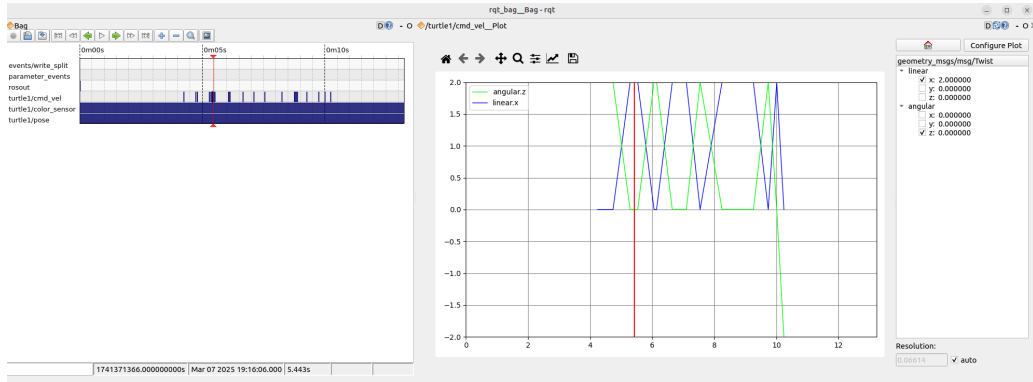
Rys. 2.13: GUI dla narzędzia RQT_TF_TREE

2.14 RQT_BAG

Graficzne narzędzie do nagrywania i wizualizacji danych zapisanych w `rosbag2`. Pozwala na analizę danych historycznych.

Uruchomienie narzędzia:

```
1 ros2 run rqt_bag rqt_bag
```



Rys. 2.14: GUI dla narzędzia RQT_BAG

3 Wizualizacja danych - RViz

RViz (ROS Visualization) to narzędzie do wizualizacji danych w ROS2. Umożliwia wyświetlanie informacji z różnych sensorów, analizę transformacji układów współrzędnych (`tf`), wizualizację map, trajektorii oraz modeli robotów. Narzędzie wykorzystywane jest m.in. do testowania algorytmów percepcji, nawigacji i sterowania.

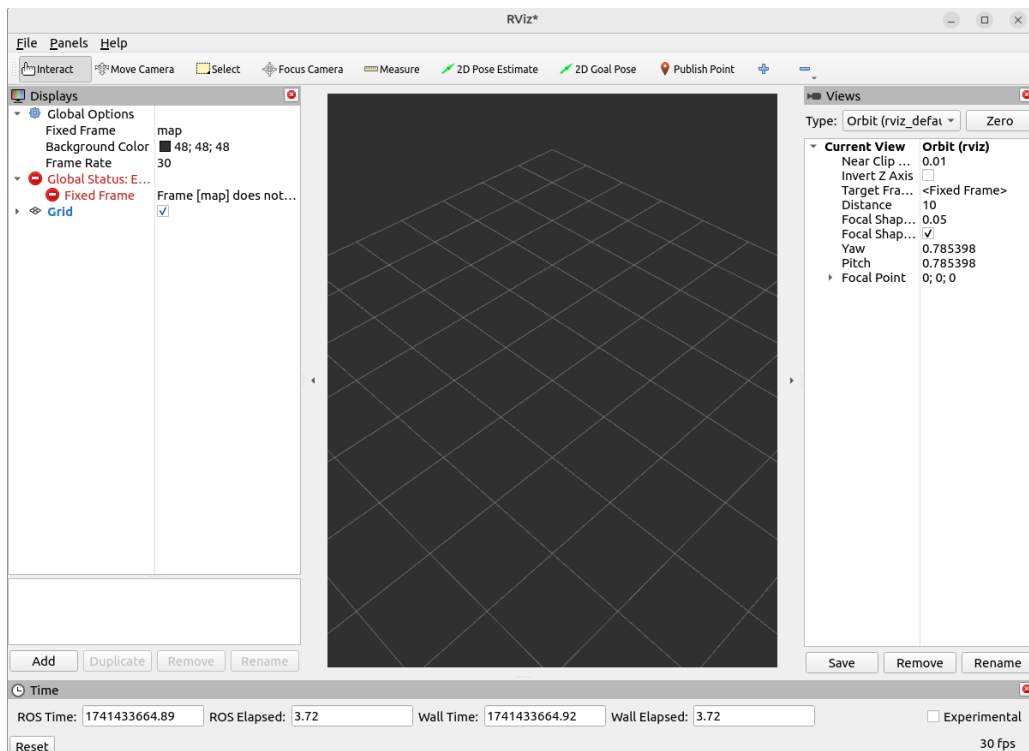
Podstawowe funkcjonalności RViz:

- Wyświetlanie chmur punktów – np. dane z LIDAR-a w formacie `sensor_msgs/PointCloud2`.
- Podgląd obrazów z kamer – np. z tematu `/camera/image_raw`.
- Wizualizacja układów współrzędnych. Wyświetlanie ramek `tf` w celu weryfikacji poprawności działania transformacji.
- Obsługa map i trajektorii (np. wizualizacja `nav_msgs/Path`). Znajduje zastosowanie podczas testów algorytmów SLAM i nawigacji.
- Podgląd położenia robota i innych obiektów - dane w formacie `geometry_msgs/PoseStamped`

Uruchamianie RViz:

```
1 ros2 run rviz2 rviz2
```

Następnie pojawia się oko:



Rys. 3.1: Okno RViz

RViz uruchomi się z domyślną konfiguracją. Można także załadować własny plik konfiguracyjny:

```
1 ros2 run rviz2 rviz2 -d my_config.rviz
```

4 Wizualizacja i Analiza Danych - PlotJuggler

PlotJuggler to narzędzie graficzne przeznaczone do analizy i wizualizacji danych numerycznych w ROS2. Umożliwia ono dynamiczne wyświetlanie i przetwarzanie danych strumieniowych.

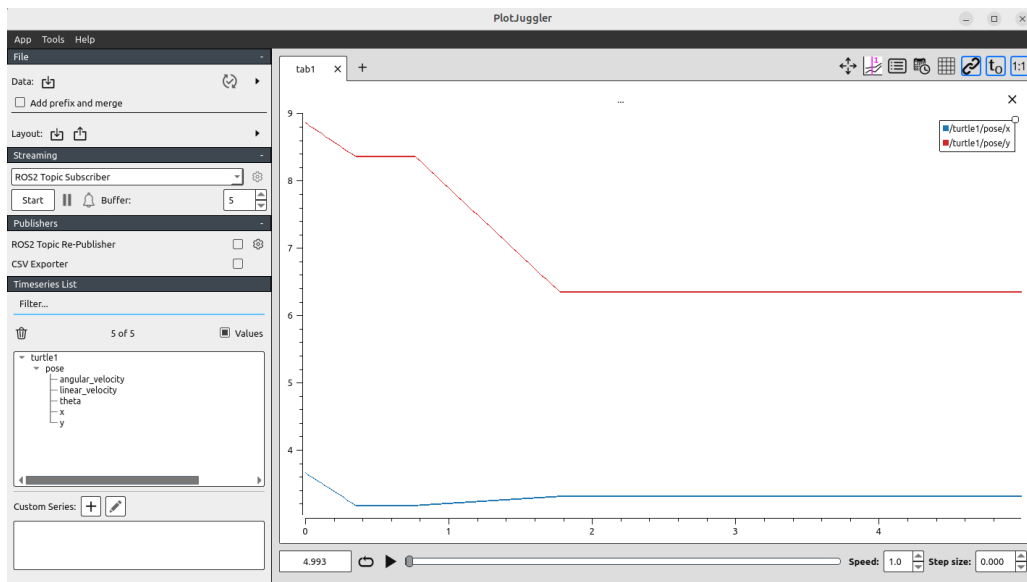
Podstawowe funkcjonalności PlotJuggler:

- Interaktywne wykresy – możliwość wyświetlania i porównywania różnych zmiennych w czasie rzeczywistym.
- Odczyt danych z ROS2 – PlotJuggler może subskrybować tematy ROS2 i wyświetlać ich wartości.
- Analiza historyczna – obsługa plików rosbag2, co pozwala na analizę wcześniejszych zapisów danych.
- Filtrowanie i przetwarzanie danych – umożliwia transformację strumieni danych bez konieczności modyfikacji kodu.
- Eksport danych – możliwość zapisywania wykresów oraz eksportowania danych do formatów takich jak CSV.

Uruchamianie PlotJuggler:

```
1 ros2 run plotjuggler plotjuggler
```

Po uruchomieniu pojawia się następujące okno:



Rys. 4.1: Główne okno PlotJuggler

5 Zegary

Jednym z wyzwań przed jakimi stoją węzły jest problem synchronizacji danych wynikający z opóźnień sprzętowych. Zastosowanie abstrakcyjnego czasu umożliwia manipulowanie nim poprzez przyspieszanie, zwalnianie i wstrzymywanie go w celu debugowania.

Domyślnym źródłem czasu w ROS jest czas systemowy z komputera. Niestety w przypadku, gdy chcemy odtwarzać dane w czasie przyspieszonym, spowolnionym lub mieć większą kontrolę nad czasem konieczne jest zastosowanie symulowanego zegara czasu. W przypadku odtwarzania danych z czujnika ważniejsze jest by timestamp z jakim są zapisane dane odnosił się do czasu z jakim były one zbierane, a nie czasem systemu komputera na którym będą odtwarzane dane np. położenie enkodera w danych chwilach czasowych. W tej sytuacji dla różnych zegarów czasu wyznaczone prędkości obrotowe na podstawie zebranych danych byłyby różne, gdyby dla tych samych danych odnosił się do nich inny timestamp.

5.1 Rodzaje zegarów

ROS2 oferuje trzy główne typy zegarów. Oto one:

- RCL_SYSTEM_TIME (Czas systemowy)
 - Używa zegara systemowego komputera.
 - Nie nadaje się do symulacji, gdzie czas może być spowolniony lub przyspieszony.
- RCL_ROS_TIME (Czas ROS)
 - Jeśli nie jest dostępne źródło czasu ROS, domyślnie używa czasu systemowego (RCL_SYSTEM_TIME).
 - Jeśli jest aktywny to zwraca ostatnią wartość zraportowaną przez źródło czasu.
 - Ustawienie parametru `use_sim_time` w węźle powoduje, że ten czas jest aktywny.
- RCL_STEADY_TIME

- Znajduje zastosowanie w sterownikach urządzeń, które komunikują się z nimi i wykorzystują timeout.

Zakłada się, że domyślnie wybieranym zegarem będzie `ROS_TIME`, jednakże możliwa jest równoległa implementacja innych zegarów zależnie od potrzeb. Wykorzystanie różnych zegarów powinno odbywać się co najwyżej w obrębie danego węzła a dane wyjściowe powinny zawierać `ROS TIME`.

5.2 Źródło czasu

Abstrakcyjny czas publikowany jest na temacie `clock`, który zawiera informację o najbardziej aktualnym czasie. Istniejący Publisher wysyłający nowe dane na ten temat spowoduje nadpisanie czasu `ROS Time`. W danej chwili tylko jedno źródło czasu może publikować informację na `clock`. Wartość czasu wynosząca 0 jest traktowana jako brak inicjalizacji czasu. Częstotliwość publikowania czasu zależna jest od aplikacji i nie ma określonej standardowej wartości. Domyślnie nie są dostępne zaawansowane metody umożliwiające estymację czasu pomiędzy kolejnymi tikami.

Jeśli mamy dostęp do innego lepszego zegara czasu do można go zastosować i odczytywać z niego wartości a następnie wysłać do `ROS`. Na przykład zegar czasu może być odczytywany z sygnału GPS. Zalecana jest integracja źródła czasu wykorzystująca standard `NTP` do synchronizacji czasu.

5.3 Implementacja

Dla zegarów dostępne jest API `SystemTime`, `SteadyTime` i `ROSTime`. Dla `SystemTime` i `ROSTime` API jest wymienne a `SteadyTime` ma już inne API.

Dla wszystkich zegarów dostępne są następujące typy danych:

- `Time`
- `Duration`
- `Rate`

6 Nagrywanie i odtwarzanie danych - Bag

Jest to format danych przechowujący informacje o wiadomościach, które zostały wybrane do zarejestrowania w trakcie działania jakiegoś procesu np. monitorowanie zmiany położenia w czasie, zbieranie danych z czujników. Zebrane dane mogą być później użyte do testowania różnych algorytmów np. do wyznaczania odometrii na podstawie fuzji danych z sensorów, testowania różnych parametrów algorytmów do mapowania terenu.

Do utworzenia bag'a wykorzystywane jest narzędzie `ros2 bag` umożliwiające rejestrowanie wiadomości `ROS`. Takie podejście umożliwia odtwarzanie i dostarczanie danych do systemu w taki sam sposób w jaki pojawiały się w trakcie działania systemu].

6.1 Rejestrowanie danych - ros2 bag record

6.1.1 Podstawowe polecenia

Rejestrowane mogą być nie tylko wszystkie dane jednocześnie tak ja się pojawiają w systemie, ale mamy możliwość wybrania sposobu jak mają być zarejestrowane. Można zarejestrować m.in.:

- pojedyncze wiadomości (oddzielne nazwy tematów)

```
1 ros2 bag record /topic_name
```

- wszystkie wiadomości (-a lub --all)

```
1 ros2 bag record --all
```

- бага o określonym czasie trwania (-d lub --max-bag-duration). Jeśli czas jest ustawiony następuje automatyczny podział. Czas trwania podawany jest w sekundach.

```
1 ros2 bag record --max-bag-duration 20 --all
```

- бага o określonym rozmiarze (-b lub --max-bag-size). Jeśli rozmiar jest ustawiony następuje automatyczny podział бага. Rozmiar pliku podawany jest w bajtach i musi być większy niż 86016.

```
1 ros2 bag record --max-bag-size 86017 --all
```

- бага w określonej lokalizacji i z podaną nazwą podają ścieżkę a na końcu nazwę бага (-o lub --output)

```
1 ros2 bag record -o /path_to_bag_storage/bag_name --all
```

Podczas rejestrowania бага, gdy ustawiony jest jednocześnie maksymalny czas trwania i rozmiar бага podział natępuje wtedy, gdy jedna z tych wartości zostanie przekroczona.

Podczas rejestrowania danych narzędzie `ros2 bag record` nie ma natywnego wsparcia dla usuwania starszych plików.

6.1.2 Zastosowanie filtracji tematów przy rejestrowaniu tematów

W ROS2 można używać wyrażeń regularnych (regex) do filtrowania tematów podczas nagrywania danych za pomocą `ros2 bag record`. Jest to przydatne, gdy chcemy nagrywać tylko określone tematy pasujące do wzorca zamiast wszystkich dostępnych.

Aby nagrywać tylko tematy spełniające określony wzorzec, należy użyć `--regex`:

```
1 ros2 bag record --regex "/camera/.*"
```

Ten zapis nagra wszystkie tematy zaczynające się od `/camera/`, np.:

- `/camera/image_raw`
- `/camera/depth`
- `/camera/infra1`

Aby nagrać kilka grup tematów, należy użyć `|` (alternatywa OR):

```
1 ros2 bag record --regex "/(camera|lidar)/.*"
```

Nagrywa wszystkie tematy, które zaczynają się od /camera/ lub /lidar/, np.:

- /camera/image_raw
- /camera/depth
- /lidar/scan
- /lidar/points

Aby nagrać tylko tematy, które kończą się np. na _raw, należy użyć:

```
1 ros2 bag record --regex ".*_raw$"
```

Nagrywa tematy takie jak:

- /camera/image_raw
- /sensor/data_raw
- /lidar/scan_raw

Wykluczanie tematów przy nagrywaniu. Nagrywanie wszystkiego oprócz tematów rozpoczynających się od /tf i /clock

```
1 ros2 bag record -a --exclude "/tf|/clock"
```

6.2 Podstawowe informacje o bagu - ros2 bag info

Polecenie `ros2 bag info` służy do uzyskiwania szczegółowych informacji o pliku rosbag2, który zawiera zapisane dane z tematów ROS2. Pozwala sprawdzić m.in. listę zapisanych tematów, liczbę wiadomości, rozmiar бага, okres nagrywania oraz format zapisu.

Składnia podstawowa polecenia:

```
1 ros2 bag info <ścieżka_do_pliku_rosbag>
```

Przykład użycia polecenia (Rys. 6.1):

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 bag info rosbag2_2025_03_08-11_44_44
Files:          rosbag2_2025_03_08-11_44_44_0.db3
Bag size:       98.8 MiB
Storage id:     sqlite3
Duration:       4.511396698s
Start:          Mar  8 2025 11:44:44.575793667 (1741430684.575793667)
End:            Mar  8 2025 11:44:49.087190365 (1741430689.087190365)
Messages:      989
Topic information: Topic: /camera/camera_info | Type: sensor_msgs/msg/CameraInfo | Count: 105 | Serialization Format: cdr
                  Topic: /camera/image_raw | Type: sensor_msgs/msg/Image | Count: 105 | Serialization Format: cdr
                  Topic: /camera/image_raw/theora | Type: theora_image_transport/msg/Package | Count: 104 | Serialization Format: cdr
                  Topic: /camera/image_raw/compressed | Type: sensor_msgs/msg/CompressedImage | Count: 105 | Serialization Format: cdr
                  Topic: /turtle1/pose | Type: turtlesim/msg/Pose | Count: 284 | Serialization Format: cdr
                  Topic: /camera/image_raw/compressedDepth | Type: sensor_msgs/msg/CompressedImage | Count: 0 | Serialization Format: cdr
                  Topic: /turtle1/cmd_vel | Type: geometry_msgs/msg/Twist | Count: 0 | Serialization Format: cdr
                  Topic: /turtle1/color_sensor | Type: turtlesim/msg/Color | Count: 286 | Serialization Format: cdr
```

Rys. 6.1: Przykładowy wynik działania polecenia `ros2 bag info /ścieżka/nazwa_baga`

6.3 Odtwarzanie бага - ros2 bag play

Polecenie `ros2 bag play` służy do odtwarzania wcześniej nagranych danych zapisanych w plikach rosbag2.

Składnia podstawowa polecenia:

```
1 ros2 bag play <ścieżka_do_pliku_rosbag>
```

Przykład użycia dla odtwarzania бага my_bag:

```
1 ros2 bag play ~/rosbags/my_bag
```

Odtwarzanie w przyspieszonym tempie:

```
1 ros2 bag play my_bag --rate 2.0
```

- `-rate 2.0` – odtwarzanie dwukrotnie szybciej niż oryginalna prędkość.
- `-rate 0.5` – odtwarzanie dwa razy wolniej.

Pominięcie początkowych X sekund i rozpoczęcie odtwarzania od danego momentu:

```
1 ros2 bag play my_bag --start-offset 10
```

Nieskończone powtarzanie odtwarzania бага w pętli:

```
1 ros2 bag play my_bag --loop
```

Odtwarzanie tylko wybranych tematów (np. `/camera/image_raw` i `/odom`)

```
1 ros2 bag play my_bag --topics /camera/image_raw /odom
```

7 Logowanie danych diagnostycznych - Logging

Moduł Logging w ROS2 służy do zarządzania logowaniem wiadomości w systemie ROS, umożliwiając monitorowanie działania węzłów, diagnozowanie błędów oraz analizę działania systemu. ROS2 wykorzystuje pozwalając na logowanie na różnych poziomach szczegółowości

- `DEBUG` – szczegółowe informacje do analizy działania systemu.
- `INFO` – standardowe komunikaty informacyjne.
- `WARN` – ostrzeżenia o potencjalnych problemach.
- `ERROR` – poważniejsze błędy wymagające uwagi.
- `FATAL` – krytyczne błędy powodujące awarię systemu.

Fragment kodu w Pythonie przedstawiający w jaki sposób logować błędy na odpowiednim poziomie:

```
1 node.get_logger().debug('My log debug message')
2 node.get_logger().info('My log info message')
3 node.get_logger().warning('My log warn message')
4 node.get_logger().error('My log error message')
5 node.get_logger().fatal('My log fatal message')
```

Zalogowanie informacji tylko przy pierwszym wywołaniu linii:

```
1 node.get_logger().info(f'Log only first msg', once=True)
```

Logowanie informacji od każdego kolejnego wywołania linii z wyjątkiem pierwszego wywołania:

```
1 node.get_logger().warning('Skip first msg', skip_first=True)
```

8 Dokumentacja i inne linki

Rozdział może zawierać dokumentację poleceń, linki do źródeł na podstawie, których powstała instrukcja oraz inne linki pozwalające poszerzyć wiedzę.

1. Dokumentacja ROS Humble

<https://docs.ros.org/en/humble/index.html>

2. Więcej informacji o czasie:

https://design.ros2.org/articles/clock_and_time.html

3. Zastosowanie narzędzi RQT dla symulacji Turtlesim:

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/Introducing-Turtlesim.html>

4. RQT_CONSOLE

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Using-Rqt-Console/Using-Rqt-Console.html>

5. Więcej o RViz <https://docs.ros.org/en/humble/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html>

6. ros2 bag

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html>

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 bag --help
usage: ros2 bag [-h] Call `ros2 bag <command> -h` for more detailed usage. ...

Various rosbag related sub-commands

options:
  -h, --help            show this help message and exit

Commands:
  convert  Given an input bag, write out a new bag with different settings
  info     Print information about a bag to the screen
  list     Print information about available plugins to the screen
  play     Play back ROS data from a bag
  record   Record ROS data to a bag
  reindex  Reconstruct metadata file for a bag

Call `ros2 bag <command> -h` for more detailed usage.
```

Rys. 8.1: Dokumentacja polecenia ros2 bag

7. ros2 bag record

```
Record ROS data to a bag

positional arguments:
  topics                List of topics to record.

options:
  -h, --help            show this help message and exit
  -a, --all             Record all topics. Required if no explicit topic list or regex filters.
  -e REGEX, --regex REGEX
                        Record only topics containing provided regular expression. Applies on top of topics list.
  -x EXCLUDE, --exclude EXCLUDE
                        Exclude topics containing provided regular expression. Works on top of --all, --regex, or topics list.
  --include-unpublished-topics
                        Discover and record topics which have no publisher. Subscriptions on such topics will be made with default QoS unless
                        otherwise specified in a QoS overrides file.
  --include-hidden-topics
                        Discover and record hidden topics as well. These are topics used internally by ROS 2 implementation.
  -o OUTPUT, --output OUTPUT
                        destination of the bagfile to create, defaults to a timestamped folder in the current directory
  -s {my_test_plugin,sqlite3}, --storage {my_test_plugin,sqlite3}
                        storage identifier to be used, defaults to 'sqlite3'
  -f {s,a}, --serialization-format {s,a}
                        rmw serialization format in which the messages are saved, defaults to the rmw currently in use
  --no-discovery        disables topic auto discovery during recording: only topics present at startup will be recorded
  -p POLLING_INTERVAL, --polling-interval POLLING_INTERVAL
                        time in ms to wait between querying available topics for recording. It has no effect if --no-discovery is enabled.
  -b MAX_BAG_SIZE, --max-bag-size MAX_BAG_SIZE
                        maximum size in bytes before the bagfile will be split. Default it is zero, recording written in single bagfile and
                        splitting is disabled.
  -d MAX_BAG_DURATION, --max-bag-duration MAX_BAG_DURATION
                        maximum duration in seconds before the bagfile will be split. Default is zero, recording written in single bagfile and
                        splitting is disabled. If both splitting by size and duration are enabled, the bag will split at whichever threshold is
                        reached first.
  --max-cache-size MAX_CACHE_SIZE
                        maximum size (in bytes) of messages to hold in each buffer of cache. Default is 100 mebibytes. The cache is handled through
                        double buffering, which means that in pessimistic case up to twice the parameter value of memory is needed. A rule of thumb
                        is to cache an order of magnitude corresponding to about one second of total recorded data volume. If the value specified is
                        0, then every message is directly written to disk.
  --compression-mode {none,file,message}
                        Determine whether to compress by file or message. Default is 'none'.
  --compression-format {fake_comp,zstd}
                        Specify the compression format/algorithm. Default is none.
  --compression-queue-size COMPRESSION_QUEUE_SIZE
                        Number of files or messages that may be queued for compression before being dropped. Default is 1.
  --compression-threads COMPRESSION_THREADS
                        Number of files or messages that may be compressed in parallel. Default is 0, which will be interpreted as the number of
                        CPU cores.
  --snapshot-mode       Enable snapshot mode. Messages will not be written to the bagfile until the "/rosbag2_recorder/snapshot" service is
                        called.
  --ignore-leaf-topics  Ignore topics without a publisher.
  --qos-profile-overrides-path QOS_PROFILE_OVERRIDES_PATH
                        Path to a yaml file defining overrides of the QoS profile for specific topics.
  --storage-preset-profile STORAGE_PRESET_PROFILE
                        Select a configuration preset for storage. This flag settings can still be overridden by corresponding settings in the
                        config passed with --storage-config-file.
  --storage-config-file STORAGE_CONFIG_FILE
                        Path to a yaml file defining storage specific configurations. For the default storage plugin settings are specified
                        through syntax: write: pragmas: ["<setting_name>" = <setting_value>] For a list of sqlite3 settings, refer to sqlite3
                        documentation
  --start-paused        Start the recorder in a paused state.
  --use-sim-time        Use simulation time for message timestamps by subscribing to the /clock topic. Until first /clock message is received, no
                        messages will be written to bag.
  --log-level {debug,info,warn,error,fatal}
                        Logging level.
```

Rys. 8.2: Dokumentacja polecenia `ros2 bag record`

8. `ros2 bag info`

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 bag info --help
usage: ros2 bag info [-h] [-s {sqlite3,my_read_only_test_plugin,my_test_plugin}] bag_path

Print information about a bag to the screen

positional arguments:
  bag_path            Bag to open

options:
  -h, --help            show this help message and exit
  -s {sqlite3,my_read_only_test_plugin,my_test_plugin}, --storage {sqlite3,my_read_only_test_plugin,my_test_plugin}
                        Storage implementation of bag. By default attempts to detect automatically - use this argument to override.
```

Rys. 8.3: Dokumentacja polecenia `ros2 bag info`

9. `ros2 bag play`

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 bag play --help
usage: ros2 bag play [-h] [-s {my_test_plugin,sqlite3,my_read_only_test_plugin}] [--read-ahead-queue-size READ_AHEAD_QUEUE_SIZE] [-r RATE]
                    [--topics TOPICS [TOPICS ...]] [--qos-profile-overrides-path QOS_PROFILE_OVERRIDES_PATH] [-l] [--remap REMAP [REMAP ...]]
                    [--storage-config-file STORAGE_CONFIG_FILE] [--clock [CLOCK]] [--d DELAY] [--disable-keyboard-controls] [-p]
                    [--start-offset START_OFFSET] [--wait-for-all-acked TIMEOUT] [--disable-loan-message]
                    [--log-level {debug,info,warn,error,fatal}]
                    bag_path

Play back ROS data from a bag

positional arguments:
  bag_path              Bag to open

options:
  -h, --help            show this help message and exit
  -s {my_test_plugin,sqlite3,my_read_only_test_plugin}, --storage {my_test_plugin,sqlite3,my_read_only_test_plugin}
                        Storage implementation of bag. By default attempts to detect automatically - use this argument to override.
  --read-ahead-queue-size READ_AHEAD_QUEUE_SIZE
                        size of message queue rosbag tries to hold in memory to help deterministic playback. Larger size will result in larger
                        memory needs but might prevent delay of message playback.
  -r RATE, --rate RATE  rate at which to play back messages. Valid range > 0.0.
  --topics TOPICS [TOPICS ...]
                        topics to replay, separated by space. If none specified, all topics will be replayed.
  --qos-profile-overrides-path QOS_PROFILE_OVERRIDES_PATH
                        Path to a yaml file defining overrides of the QoS profile for specific topics.
  -l, --loop            enables loop playback when playing a bagfile: it starts back at the beginning on reaching the end and plays indefinitely.
  --remap REMAP [REMAP ...], -m REMAP [REMAP ...]
                        list of topics to be remapped: in the form "old_topic1:=new_topic1 old_topic2:=new_topic2 etc."
  --storage-config-file STORAGE_CONFIG_FILE
                        Path to a yaml file defining storage specific configurations. For the default storage plugin settings are specified
                        through syntax:read: pragmas: ["<setting_name>" = <setting_value>]Note that applicable settings are limited to read-only
                        for ros2 bag play.For a list of sqlite3 settings, refer to sqlite3 documentation
  --clock [CLOCK]       Publish to /clock at a specific frequency in Hz, to act as a ROS Time Source. Value must be positive. Defaults to not
                        publishing.
  -d DELAY, --delay DELAY
                        Sleep duration before play (each loop), in seconds. Negative durations invalid.
  --disable-keyboard-controls
                        disables keyboard controls for playback
  -p, --start-paused    Start the playback player in a paused state.
  --start-offset START_OFFSET
                        Start the playback player this many seconds into the bag file.
  --wait-for-all-acked TIMEOUT
                        Wait until all published messages are acknowledged by all subscribers or until the timeout elapses in millisecond before
                        play is terminated. Especially for the case of sending message with big size in a short time. Negative timeout is invalid.
                        0 means wait forever until all published messages are acknowledged by all subscribers. Note that this option is valid only
                        if the publisher's QoS profile is RELIABLE.
  --disable-loan-message
                        Disable to publish as loaned message. By default, if loaned message can be used, messages are published as loaned message.
                        It can help to reduce the number of data copies, so there is a greater benefit for sending big data.
  --log-level {debug,info,warn,error,fatal}
                        Logging level.
```

Rys. 8.4: Dokumentacja polecenia ros2 bag play

10. Więcej o Logging i jego konfiguracji

<https://docs.ros.org/en/rolling/Tutorials/Demos/Logging-and-logger-configuration.html>